

AT32F435/437 DMA User Guide

Introduction

The AT32F435/437 series MCUs are designed with two DMA controllers (DMA1 and DMA2); each supports seven channels and manages memory access requests from one or more peripherals. This application note mainly introduces main features and application cases of DMA controller.

Applicable products:

Part number	AT32F435 series
	AT32F437 series

Contents

1	DMA controller	6
2	DMA multiplexer (DMAMUX)	7
3	DMA functions.....	9
3.1	Programmable data transfer width.....	9
3.2	DMAMUX configuration	9
3.3	Request generator module	10
3.4	Request synchronization module.....	10
4	DMA configuration.....	11
4.1	Functions.....	11
4.2	Data stream.....	11
4.3	Configuration process.....	12
5	Example 1: Data transfer from FLASH to SRAM	13
5.1	Introduction.....	13
5.2	Resources	13
5.3	Software design.....	13
5.4	Test result.....	14
6	Example 2: TMR generates a DMA request for data transfer from SRAM to	
GPIO	15	
6.1	Introduction.....	15
6.2	Resources	15
6.3	Software design.....	15
6.4	Test result.....	17
7	Example 3: DMA request generator generates DMA requests.....	18
7.1	Introduction.....	18
7.2	Resources	18

7.3	Software design.....	18
7.4	Test result.....	20
8	Example 4: DMA data transfer after reception of DMA request synchronized signal	21
8.1	Introduction.....	21
8.2	Resources	21
8.3	Software design.....	21
8.4	Test result.....	23
9	Revision history.....	24

List of tables

Table 1. Flexible DMA1/DMA2 request mapping.....	7
Table 2. Channel configuration functions.....	11
Table 3. Document revision history.....	24

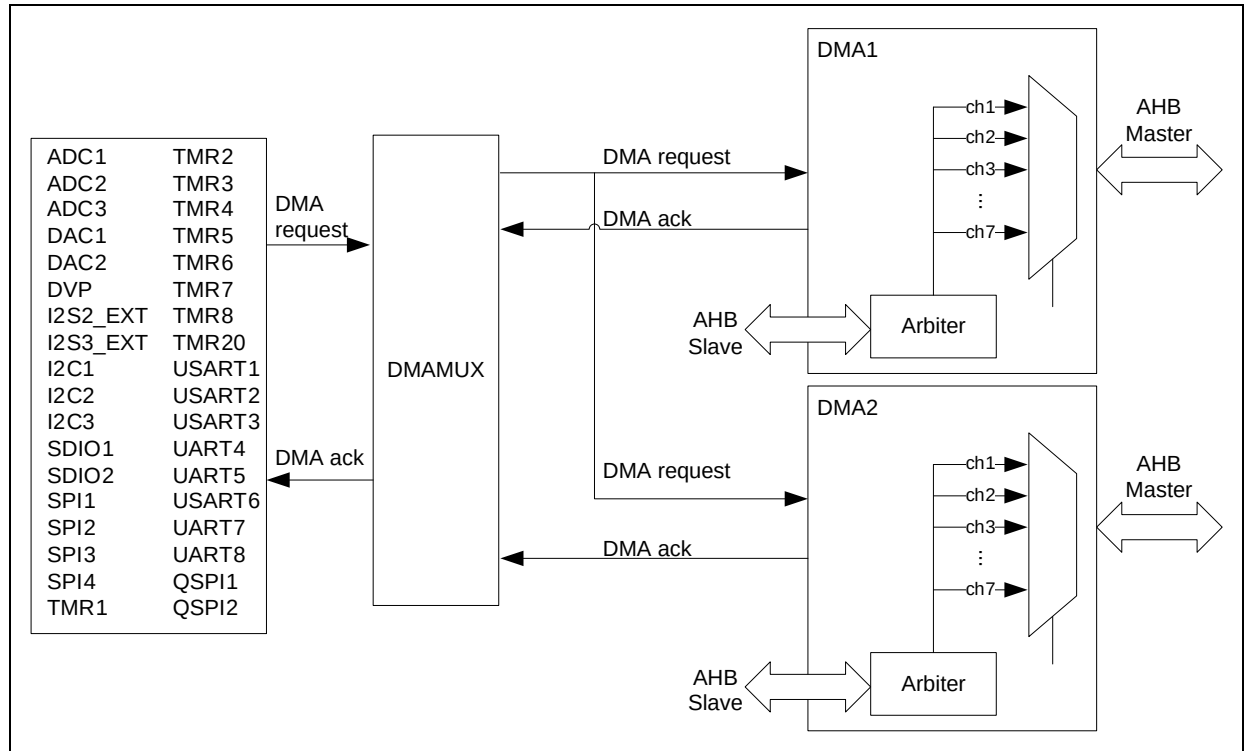
List of figures

Figure 1. DMA controller block diagram	6
Figure 2. PWIDTH: byte, MWIDTH: half-word.....	9
Figure 3. PWIDTH: half-word, MWIDTH: word.....	9
Figure 4. Test result of data transfer to GPIO0-GPIO3	17

1 DMA controller

Direct memory access (DMA) controller is designed for 32-bit MCU applications with the aim of enhancing system performance and reducing the generation of interrupts. One DMA controller contains seven channels for memory-to-memory, memory-to-peripheral and peripheral-to-memory transfers, and each channel manages memory access requests from one or more peripherals.

Figure 1. DMA controller block diagram



2 DMA multiplexer (DMAMUX)

DMAMUX is required for peripheral DMA request mapping on a data stream channel. It designs a unique ID for each peripheral, and users only need to write this ID into the corresponding register and enable DMAMUX. The use of DMAMUX makes DMA more flexible, and users can assign the even channels as needed, not considering that a DMA request is only used for one or more dedicated channels.

Table 1 lists the request ID of each IP.

Table 1. Flexible DMA1/DMA2 request mapping

DMAMUX request	Source	DMAMUX request	Source	DMAMUX request	Source	DMAMUX request	Source
1	DMA_MUXREQG1	33	UART5_TX	65	TMR3_OVERFLOW	97	reserved
2	DMA_MUXREQG2	34	reserved	66	TMR3_TRIG	98	reserved
3	DMA_MUXREQG3	35	reserved	67	TMR4_CH1	99	reserved
4	DMA_MUXREQG4	36	ADC2	68	TMR4_CH2	100	reserved
5	ADC1	37	ADC3	69	TMR4_CH3	101	reserved
6	DAC1	38	reserved	70	TMR4_CH4	102	reserved
7	reserved	39	SDIO1	71	TMR4_UP	103	SDIO2
8	TMR6_OVERFLOW	40	QSPI1	72	TMR5_CH1	104	QSPI2
9	TMR7_OVERFLOW	41	DAC2	73	TMR5_CH2	105	DVP
10	SPI1_RX	42	TMR1_CH1	74	TMR5_CH3	106	SPI4_RX
11	SPI1_TX	43	TMR1_CH2	75	TMR5_CH4	107	SPI4_TX
12	SPI2_RX	44	TMR1_CH3	76	TMR5_OVERFLOW	108	reserved
13	SPI2_TX	45	TMR1_CH4	77	TMR5_TRIG	109	reserved
14	SPI3_RX	46	TMR1_OVERFLOW	78	reserved	110	I2S2_EXT_RX
15	SPI3_TX	47	TMR1_TRIG	79	reserved	111	I2S2_EXT_TX
16	I2C1_RX	48	TMR1_HALL	80	reserved	112	I2S3_EXT_RX
17	I2C1_TX	49	TMR8_CH1	81	reserved	113	I2S3_EXT_TX
18	I2C2_RX	50	TMR8_CH2	82	reserved	114	USART6_RX
19	I2C2_TX	51	TMR8_CH3	83	reserved	115	USART6_TX
20	I2C3_RX	52	TMR8_CH4	84	reserved	116	UART7_RX
21	I2C3_TX	53	TMR8_OVERFLOW	85	reserved	117	UART7_TX
22	reserved	54	TMR8_TRIG	86	TMR20_CH1	118	UART8_RX
23	reserved	55	TMR8_HALL	87	TMR20_CH2	119	UART8_TX
24	USART1_RX	56	TMR2_CH1	88	TMR20_CH3	120	reserved
25	USART1_TX	57	TMR2_CH2	89	TMR20_CH4	121	reserved
26	USART2_RX	58	TMR2_CH3	90	TMR20_OVERFLOW	122	reserved
27	USART2_TX	59	TMR2_CH4	91	reserved	123	reserved
28	USART3_RX	60	TMR2_OVERFLOW	92	reserved	124	reserved
29	USART3_TX	61	TMR3_CH1	93	TMR20_TRIG	125	reserved
30	UART4_RX	62	TMR3_CH2	94	TMR20_HALL	126	TMR2_TRIG

DMAMUX request	Source	DMAMUX request	Source	DMAMUX request	Source	DMAMUX request	Source
31	UART4_TX	63	TMR3_CH3	95	reserved	127	reserved
32	UART5_RX	64	TMR3_CH4	96	reserved		

Note: The “DMAMUX request” is the ID, and “source” is the DMA request.

3 DMA functions

3.1 Programmable data transfer width

DMA controller channels support data transfer width in byte/half-word/word. The transfer widths of the source data and destination data are programmable by setting the PWIDTH and MWIDTH bits in the DMA_CxCTRL register. When PWIDTH is not equal to MWIDTH, it can be aligned according to the settings of PWIDTH/ MWIDTH.

Figure 2. PWIDTH: byte, MWIDTH: half-word

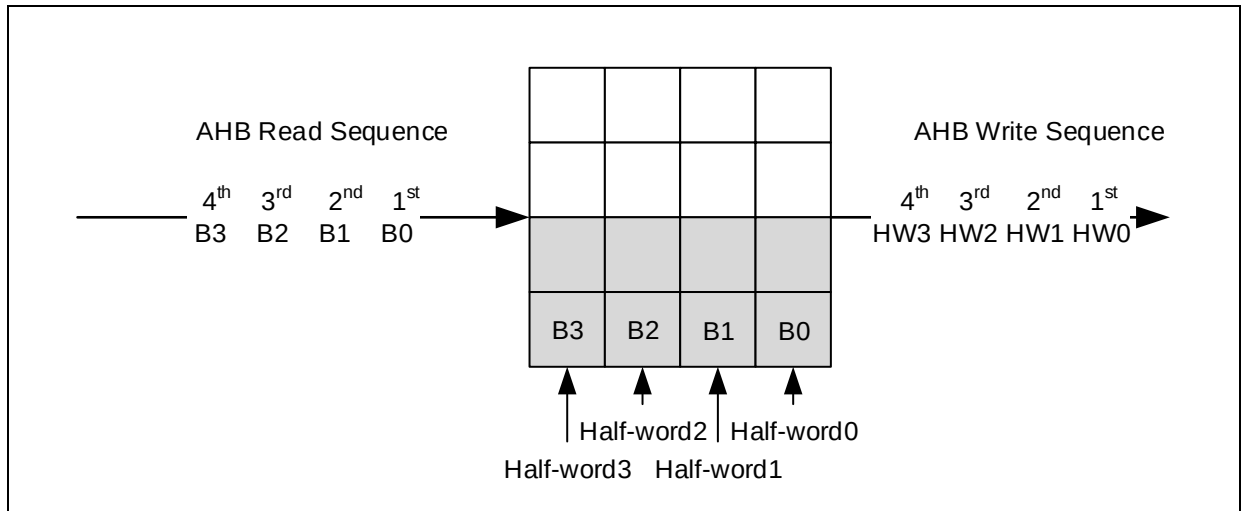
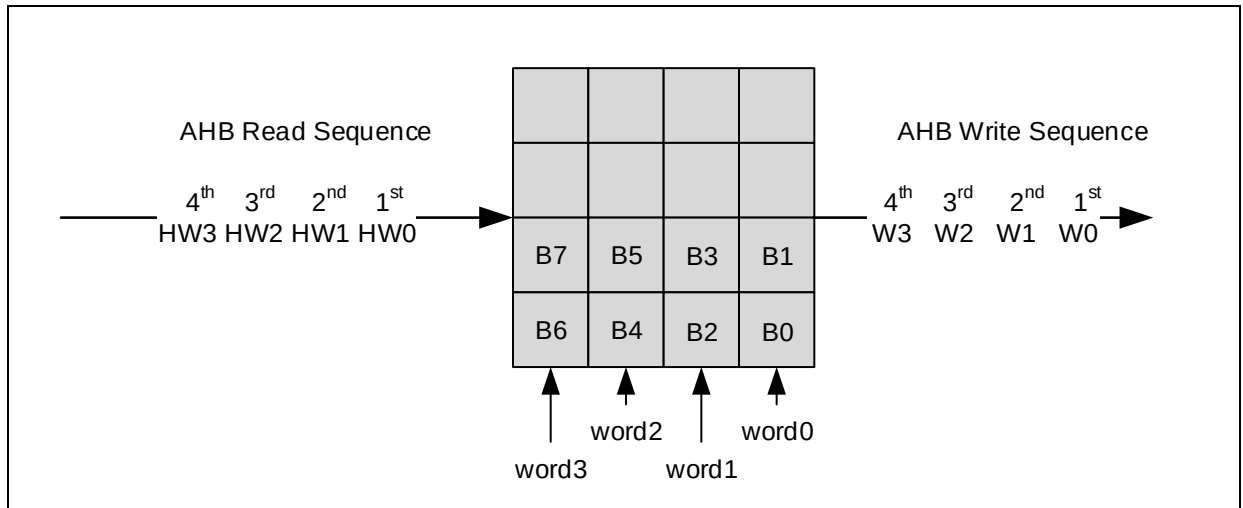


Figure 3. PWIDTH: half-word, MWIDTH: word



3.2 DMAMUX configuration

In M2P and P2M modes, the DMAMUX must be configured properly so that the DMA controller can respond to peripheral DMA request. The DMAMUX is used to multiplex channels for peripheral DMA request, that is, any peripheral DMA request can be mapped on any channel of DMA1/DMA2, which greatly improves flexibility of channel allocation.

Call the following two dedicated functions to configure DMAMUX.

```
/* Enable DMAMUX function */
void dmamux_enable(dma_type *dma_x, confirm_state new_state);
```

```
/* Configure DMAMUX channel */  
void dmamux_init(dmamux_channel_type *dmamux_channelx, dmamux_request_id_sel_type dmamux_req_sel);
```

3.3 Request generator module

When the DMAMUX is enabled and configured, users can configure the DMA request generator module. The module contains four request generator channels, and it can use external EXINT input as DMA request source, without the needs of any traditional peripherals (such as TIMER, SPI, etc.) to generate DMA requests.

Call the following dedicated functions to configure the DMA request generator module.

```
/* Initialize request generator structure parameters */  
dmamux_generator_default_para_init(&dmamux_gen_init_struct);  
/* Configure request generator structure */  
dmamux_gen_init_struct.gen_polarity = DMAMUX_GEN_POLARITY_RISING;  
dmamux_gen_init_struct.gen_request_number = 4;  
dmamux_gen_init_struct.gen_signal_sel = DMAMUX_GEN_ID_EXINT0;  
dmamux_gen_init_struct.gen_enable = TRUE;  
/* Configure request generator */  
dmamux_generator_config(DMA2MUX_GENERATOR1, &dmamux_gen_init_struct);
```

3.4 Request synchronization module

When the DMAMUX is enabled and configured, users can configure the DMA request synchronization module that consists of seven synchronization channels. With the DMA synchronization enabled, when a peripheral generates a DMA request, the DMA does not immediately respond and transfer data, but waits for a synchronized signal. After the synchronized signal is received, DMA transfers data according to the configuration. The synchronized signal can be provided by external EXINT input.

Call the following dedicated functions to configure DMA synchronization module.

```
/* Initialize synchronization module structure parameters */  
dmamux_sync_default_para_init(&dmamux_sync_init_struct);  
/* Set synchronization module structure parameters */  
dmamux_sync_init_struct.sync_request_number = 4;  
dmamux_sync_init_struct.sync_signal_sel = DMAMUX_SYNC_ID_EXINT0;  
dmamux_sync_init_struct.sync_polarity = DMAMUX_SYNC_POLARITY_RISING;  
dmamux_sync_init_struct.sync_event_enable = FALSE;  
dmamux_sync_init_struct.sync_enable = TRUE;  
/* Configure synchronization module */  
dmamux_sync_config(DMA2MUX_CHANNEL4, &dmamux_sync_init_struct);
```

4 DMA configuration

This section mainly introduces EDMA configuration functions and process.

4.1 Functions

Table 2. Channel configuration functions

/* reset channels */ void dma_reset(dma_channel_type *dma_channel);
/* initialize DMA structure parameters */ void dma_default_para_init(dma_init_type *dma_init_struct);
/* initialize channels */ void dma_init(dma_channel_type *dma_channel, dma_init_type *dma_init_struct);
/* enable channels */ void dma_channel_enable(dma_channel_type *dma_channel, confirm_state new_state);
/* enable DMAMUX */ void dmamux_enable(dma_type *dma_x, confirm_state new_state);
/* write DMA request ID */ void dmamux_init(dmamux_channel_type *dmamux_channel, dmamux_req_id_sel_type dmamux_req_sel);

4.2 Data stream

■ Set peripheral address in the CxPADDR register

The initial peripheral address for data transfer remains unchanged during transmission.

■ Set memory address in the CxMADDR register

The initial memory address for data transfer remains unchanged during transmission.

■ Configure the amount of data to be transferred in the CxDTCNT register

Programmable data transfer size is up to 65535. This value is decremented after each data transfer.

■ Configure the channel in the CxCTRL register

It includes the channel priority, data transfer direction/width, address incremented mode, circular mode and interrupt mode.

➤ Channel priority (CHPL)

There are four levels, including very high priority, high priority, medium priority and low priority.

If the two channels have the same priority level, then the channel with lower number will get priority over the one with higher number. For example, channel 1 has priority over channel 2.

➤ Data transfer direction (DTD)

It includes memory-to-peripheral (M2P), peripheral-to-memory (P2M) and memory-to-memory (M2M).

In M2M mode, circular mode, dual memory mode and direct mode cannot be used.

➤ Data transfer width (PWIDTH/ MWIDTH)

It can be configured in byte, half-word or word.

- **Address incremented mode (PINCM/MINCM)**
In incrementing mode, the subsequent transfer address is the previous address plus transfer width (PWIDTH/MWIDTH).
- **Circular mode (LM)**
In circular mode, the contents in the CxDTCNT register is automatically reloaded with the initially programmed value after the completion of the last transfer.
- **Enable DMAMUX by setting the TBL_SEL bit in the MUXSEL register**
In non-M2M mode, the DMAMUX must be enabled to start a data stream in response to DMA requests from peripherals.
- **Write peripheral DMA request ID by setting the REQSEL bit in the MUXCxCTRL register**
In non-M2M mode, write the peripheral DMA request ID to start a data stream in response to DMA requests from peripherals.
- **Enable data stream by setting the CHEN bit in the CxCTRL register**

4.3 Configuration process

- Enable DMA clock;
- Call the channel reset function to reset data stream;
- Call the structure initialization function to initialize channel configuration structure;
- Call the initialization function to initialize channels;
- Call the DMAMUX enable function and ID write function to configure DMAMUX;
- Call the channel enable function to enable channels.

5 Example 1: Data transfer from FLASH to SRAM

5.1 Introduction

Use DMA controller to transfer data from FLASH to SRAM.

5.2 Resources

- 1) Hardware
AT-START BOARD of the corresponding MCU series
- 2) Software
project\at_start_f4xx\examples\dma\flash_to_sram

5.3 Software design

- 1) Configuration
 - Enable DMA peripheral clock
 - Configure DMA channels
 - Enable transfer completed interrupt
 - Enable channels
 - Wait for the end of data transfer
 - Check if the transmitted data and received data are equal
- 2) Code
 - Main function code

```
int main(void)
{
    /* initialize system clock */
    system_clock_config();
    /* Initialize AT-START board */
    at32_board_init();
    /* enable DMA1 clock */
    crm_periph_clock_enable(CRM_DMA1_PERIPH_CLOCK, TRUE);

    /* initialize dma1 channel */
    dma_reset(DMA1_CHANNEL1);
    dma_init_struct.buffer_size = BUFFER_SIZE;
    dma_init_struct.direction = DMA_DIR_MEMORY_TO_MEMORY;
    dma_init_struct.memory_base_addr = (uint32_t)dst_buffer;
    dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_WORD;
    dma_init_struct.memory_inc_enable = TRUE;
    dma_init_struct.peripheral_base_addr = (uint32_t)src_const_buffer;
    dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_WORD;
    dma_init_struct.peripheral_inc_enable = TRUE;
    dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
    dma_init_struct.loop_mode_enable = FALSE;
```

```
dma_init(DMA1_CHANNEL1, &dma_init_struct);

/* enable transfer completed interrupt */
dma_interrupt_enable(DMA1_CHANNEL1, DMA_FDT_INT, TRUE);

/* dma1 channel1 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(DMA1_Channel1_IRQn, 1, 0);

dma_channel_enable(DMA1_CHANNEL1, TRUE);

/* wait the end of transmission */
while(data_counter_end != 0)
{
}

/* check if the transmitted and received data are equal */
transfer_status = buffer_compare(src_const_buffer, dst_buffer, BUFFER_SIZE);

/* check if the transmitted data and received data are equal; if equal, LED lights up */
if(transfer_status == SUCCESS)
{
    /* turn led2/led3/led4 on */
    at32_led_on(LED2);
    at32_led_on(LED3);
    at32_led_on(LED4);
}

while(1)
{
}
```

5.4 Test result

- If the transmitted data and received data are equal, LED2/3/4 will be ON.

6 Example 2: TMR generates a DMA request for data transfer from SRAM to GPIO

6.1 Introduction

TMR generates a DMA request for data transfer from SRAM to GPIOC. Users can use a logic analyzer to view the waveform.

6.2 Resources

- 1) Hardware
AT-START BOARD of the corresponding MCU series
- 2) Software
project\at_start_f4xx\examples\dma\dmamux_data_to_gpio

6.3 Software design

- 1) Configuration
 - Enable DMA/TMR2/GPIOC clocks
 - Configure DMA channels
 - Configure TMR2 overflow DMA request
 - Enable transfer completed interrupt
 - Enable DMAMUX
 - Enable DMA channels
 - Enable TMR2 overflow interrupt to generate a DMA request

- 2) Code

- Main function code

```
#define BUFFER_SIZE    16
uint16_t src_buffer[BUFFER_SIZE] = {0x0001, 0x0002, 0x0003, 0x0004, 0x0005, 0x0006, 0x0007, 0x0008,
                                     0x0009, 0x000a, 0x000b, 0x000c, 0x000d, 0x000e, 0x000f, 0x0010};

Int main(void)
{
    /* initialize system clock */
    system_clock_config();
    /* Initialize AT-START board */
    at32_board_init();
    /* enable dma2/gpioc/tmr2 clocks */
    crm_periph_clock_enable(CRM_DMA2_PERIPH_CLOCK, TRUE);
    crm_periph_clock_enable(CRM_GPIOC_PERIPH_CLOCK, TRUE);
    crm_periph_clock_enable(CRM_TMR2_PERIPH_CLOCK, TRUE);

    /* config gpioc pin for output mode */
    gpio_init_struct.gpio_pins = GPIO_PINS_ALL;
    gpio_init_struct.gpio_mode = GPIO_MODE_OUTPUT;
    gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
```

```
gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
gpio_init(GPIOC, &gpio_init_struct);
/* enable TMR2 */
tmr_base_init(TMR2, 0xFF, 0);
tmr_cnt_dir_set(TMR2, TMR_COUNT_UP);

/* enable TMR2 overflow DMA request */
tmr_dma_request_enable(TMR2, TMR_OVERFLOW_DMA_REQUEST, TRUE);

/* initialize dma2 channel1 */
dma_reset(DMA2_CHANNEL1);
dma_init_struct.buffer_size = BUFFER_SIZE;
dma_init_struct.direction = DMA_DIR_MEMORY_TO_PERIPHERAL;
dma_init_struct.memory_base_addr = (uint32_t)src_buffer;
dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_HALFWORD;
dma_init_struct.memory_inc_enable = TRUE;
dma_init_struct.peripheral_base_addr = (uint32_t)&GPIOC->odr;
dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_HALFWORD;
dma_init_struct.peripheral_inc_enable = FALSE;
dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
dma_init_struct.loop_mode_enable = FALSE;
dma_init(DMA2_CHANNEL1, &dma_init_struct);
/* enable transfer completed interrupt */
dma_interrupt_enable(DMA2_CHANNEL1, DMA_FDT_INT, TRUE);

/* dma2 channel1 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(DMA2_Channel1_IRQn, 1, 0);

/* enable TMR2 DMAMUX */
dmamux_enable(DMA2, TRUE);
dmamux_init(DMA2MUX_CHANNEL1, DMAMUX_DMAREQ_ID_TMR2_OVERFLOW);

/* enable dma channel */
dma_channel_enable(DMA2_CHANNEL1, TRUE);

/* enable tmr2 */
tmr_counter_enable(TMR2, TRUE);

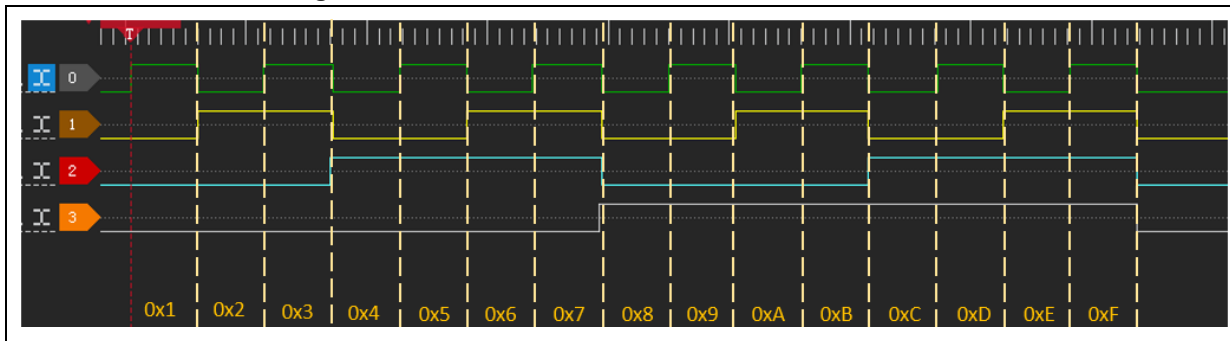
while(1)
{
}
}
```

6.4 Test result

- Use a logic analyzer to capture the GPIOC waveforms.

The captured waveform of GPIO0-GPIO3 is shown below.

Figure 4. Test result of data transfer to GPIO0-GPIO3



7 Example 3: DMA request generator generates DMA requests

7.1 Introduction

Press the on-board button, and then DMAMUX generates a DMA request to transfer data from source address to destination address.

7.2 Resources

- 1) Hardware
AT-START BOARD of the corresponding MCU series
- 2) Software
project\at_start_f4xx\examples\dma\dmamux_genertor_exint

7.3 Software design

- 1) Configuration
 - Enable DMA peripheral clocks
 - Configure DMA channels and DMAMUX request generator module
 - Enable transfer completed interrupt
 - Enable DMA channels
 - Wait for the end of data transfer
- 2) Code
 - Main function code

```
int main(void)
{
    /* initialize system clock */
    system_clock_config();

    /* Initialize AT-START board */
    at32_board_init();

    /* enable dma2/gpioa clock */
    crm_periph_clock_enable(CRM_DMA2_PERIPH_CLOCK, TRUE);
    crm_periph_clock_enable(CRM_GPIOA_PERIPH_CLOCK, TRUE);

    /* config pa1 for input mode */
    gpio_init_struct.gpio_pins = GPIO_PINS_0;
    gpio_init_struct.gpio_mode = GPIO_MODE_INPUT;
    gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
    gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
    gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
    gpio_init(GPIOA, &gpio_init_struct);
```

```
scfg_exint_line_config(SCFG_PORT_SOURCE_GPIOA, SCFG_PINS_SOURCE0);

exint_default_para_init(&exint_init_struct);
exint_init_struct.line_enable = TRUE;
exint_init_struct.line_mode = EXINT_LINE_INTERRUPT;
exint_init_struct.line_select = EXINT_LINE_0;
exint_init_struct.line_polarity = EXINT_TRIGGER_RISING_EDGE;
exint_init(&exint_init_struct);

/* exint line1 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(EXINT0_IRQn, 1, 0);

/* dma2 channel4 configuration */
dma_reset(DMA2_CHANNEL4);
dma_init_struct.buffer_size = BUFFER_SIZE;
dma_init_struct.direction = DMA_DIR_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_base_addr = (uint32_t)dst_buffer;
dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_HALFWORD;
dma_init_struct.memory_inc_enable = TRUE;
dma_init_struct.peripheral_base_addr = (uint32_t)src_buffer;
dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_HALFWORD;
dma_init_struct.peripheral_inc_enable = TRUE;
dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
dma_init_struct.loop_mode_enable = FALSE;
dma_init(DMA2_CHANNEL4, &dma_init_struct);

/* genertor1 configuration */
dmamux_generator_default_para_init(&dmamux_gen_init_struct);
dmamux_gen_init_struct.gen_polarity = DMAMUX_GEN_POLARITY_RISING;
dmamux_gen_init_struct.gen_request_number = 4;
dmamux_gen_init_struct.gen_signal_sel = DMAMUX_GEN_ID_EXINT0;
dmamux_gen_init_struct.gen_enable = TRUE;
dmamux_generator_config(DMA2MUX_GENERATOR1, &dmamux_gen_init_struct);

/* enable transfer full data interrupt */
dma_interrupt_enable(DMA2_CHANNEL4, DMA_FDT_INT, TRUE);

/* dma2 channel4 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(DMA2_Channel4_IRQn, 1, 0);

/* dmamux function enable */
dmamux_enable(DMA2, TRUE);
dmamux_init(DMA2MUX_CHANNEL4, DMAMUX_DMAREQ_ID_REQ_G1);
```

```
/* enable dma channe4 */  
dma_channel_enable(DMA2_CHANNEL4, TRUE);  
  
while(1)  
{  
}  
}
```

7.4 Test result

Each time the on-board USER button is pressed, DMA sends 4 data to the dst_buffer array, and DMA transfer completed interrupt occurs when 16 data are sent.

8 Example 4: DMA data transfer after reception of DMA request synchronized signal

8.1 Introduction

When a DMA request is generated by TMR, the data can be transferred from source address to destination address after the request synchronized signal (input by pressing on-board button) is received.

8.2 Resources

1) Hardware

AT-START BOARD of the corresponding MCU series

2) Software

project\at_start_f4xx\examples\dma\ dmamux_synchronization_exint

8.3 Software design

1) Configuration

- Enable DMA peripheral clocks
- Configure DMA channels and DMAMUX synchronization module
- Configure TMR
- Enable transfer completed interrupt
- Enable DMA channels
- Wait for the end of data transfer

2) Code

■ Main function code

```
int main(void)
{
    /* Initialize system clock */
    system_clock_config();

    /* Initialize AT-START board */
    at32_board_init();

    /* enable dma2/gpioa clock */
    crm_periph_clock_enable(CRM_DMA2_PERIPH_CLOCK, TRUE);
    crm_periph_clock_enable(CRM_GPIOA_PERIPH_CLOCK, TRUE);
    crm_periph_clock_enable(CRM_TMR1_PERIPH_CLOCK, TRUE);

    /* config pa1 for input mode */
    gpio_init_struct.gpio_pins = GPIO_PINS_0;
    gpio_init_struct.gpio_mode = GPIO_MODE_INPUT;
    gpio_init_struct.gpio_out_type = GPIO_OUTPUT_PUSH_PULL;
    gpio_init_struct.gpio_pull = GPIO_PULL_NONE;
```

```
gpio_init_struct.gpio_drive_strength = GPIO_DRIVE_STRENGTH_STRONGER;
gpio_init(GPIOA, &gpio_init_struct);

scfg_exint_line_config(SCFG_PORT_SOURCE_GPIOA, SCFG_PINS_SOURCE0);

exint_default_para_init(&exint_init_struct);
exint_init_struct.line_enable = TRUE;
exint_init_struct.line_mode = EXINT_LINE_INTERRUPT;
exint_init_struct.line_select = EXINT_LINE_0;
exint_init_struct.line_polarity = EXINT_TRIGGER_RISING_EDGE;
exint_init(&exint_init_struct);

/* exint line1 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(EXINT0_IRQn, 1, 0);

/* dma2 channel4 configuration */
dma_reset(DMA2_CHANNEL4);
dma_init_struct.buffer_size = BUFFER_SIZE;
dma_init_struct.direction = DMA_DIR_PERIPHERAL_TO_MEMORY;
dma_init_struct.memory_base_addr = (uint32_t)dst_buffer;
dma_init_struct.memory_data_width = DMA_MEMORY_DATA_WIDTH_HALFWORD;
dma_init_struct.memory_inc_enable = TRUE;
dma_init_struct.peripheral_base_addr = (uint32_t)src_buffer;
dma_init_struct.peripheral_data_width = DMA_PERIPHERAL_DATA_WIDTH_HALFWORD;
dma_init_struct.peripheral_inc_enable = TRUE;
dma_init_struct.priority = DMA_PRIORITY_MEDIUM;
dma_init_struct.loop_mode_enable = FALSE;
dma_init(DMA2_CHANNEL4, &dma_init_struct);

dmamux_sync_default_para_init(&dmamux_sync_init_struct);
dmamux_sync_init_struct.sync_request_number = 4;
dmamux_sync_init_struct.sync_signal_sel = DMAMUX_SYNC_ID_EXINT0;
dmamux_sync_init_struct.sync_polarity = DMAMUX_SYNC_POLARITY_RISING;
dmamux_sync_init_struct.sync_event_enable = FALSE;
dmamux_sync_init_struct.sync_enable = TRUE;
dmamux_sync_config(DMA2MUX_CHANNEL4, &dmamux_sync_init_struct);

/* enable transfer full data interrupt */
dma_interrupt_enable(DMA2_CHANNEL4, DMA_FDT_INT, TRUE);

/* dma2 channel4 interrupt nvic init */
nvic_priority_group_config(NVIC_PRIORITY_GROUP_4);
nvic_irq_enable(DMA2_Channel4_IRQn, 1, 0);

/* dmamux function enable */
```

```
dmamux_enable(DMA2, TRUE);
dmamux_init(DMA2MUX_CHANNEL4, DMAMUX_DMAREQ_ID_TMR1_OVERFLOW);

/* enable dma channe4 */
dma_channel_enable(DMA2_CHANNEL4, TRUE);

/* tmr1 configuration */
tmr_base_init(TMR1, 5000, 5000);
tmr_cnt_dir_set(TMR1, TMR_COUNT_UP);

/* enable tmr1 overflow edam request */
tmr_dma_request_enable(TMR1, TMR_OVERFLOW_DMA_REQUEST, TRUE);

/* enable tmr1 */
tmr_counter_enable(TMR1, TRUE);
while(1)
{
}
}
```

8.4 Test result

When TMR generate an overflow event and a DMA request, the DMA request will not be responded. Each time the on-board USER button is pressed, four TMR DMA requests are synchronized and then DMA sends data to the dst_buffer array. The DMA transfer completed interrupt occurs when 16 data are sent.

9 Revision history

Table 3. Document revision history

Date	Version	Revision note
2021.10.29	2.0.0	Initial release.
2023.03.27	2.0.1	Added application cases.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2023 Artery Technology -All Rights Reserved